

TIPOS DE DATOS Y CONSTANTES

Teoría Completa para la Sesión 7

Módulo: Diseño de Algoritmos (CSTI12010)

Capacidad 1.2: Utilizar tipos de datos, constantes, variables, operadores

Fecha: Jueves 24 de Septiembre 2026

Facilitador: Giovanni Antonio Coto Calderón

TABLA DE CONTENIDOS

1. Introducción y Objetivos
2. Concepto de Tipos de Datos
3. Variables: Declaración y Uso
4. Los 4 Tipos de Datos en PSeInt
 - 4.1 Tipo Entero
 - 4.2 Tipo Real
 - 4.3 Tipo Carácter (Cadena de texto)
 - 4.4 Tipo Lógico
5. Constantes: Definición y Características
6. Indicadores y Reglas de Nomenclatura
7. Conversión entre Tipos
8. Ejemplos Prácticos
9. Referencias

1. INTRODUCCIÓN Y OBJETIVOS

Uno de los conceptos fundamentales en programación es entender cómo se almacenan y procesan los datos. Cada valor que un programa maneja pertenece a un tipo específico, y en PSeInt, como en la mayoría de los lenguajes de programación, debemos declarar explícitamente qué tipo de datos vamos a utilizar.

Objetivos de esta sesión:

Comprender qué es un tipo de dato y por qué es importante en programación

Identificar y utilizar correctamente los 4 tipos de datos de PSeInt

Declarar variables y constantes con sus tipos correspondientes

Aplicar reglas de nomenclatura correctas en PSeInt

Convertir valores entre diferentes tipos de datos cuando sea necesario

2. CONCEPTO DE TIPOS DE DATOS

¿Qué es un tipo de dato?

Un tipo de dato es una clasificación que define el conjunto de valores que puede almacenar una variable y las operaciones que se pueden realizar con esos valores. Cada tipo de dato ocupa una cantidad específica de memoria y tiene límites en los valores que puede representar.

¿Por qué son importantes los tipos de datos?

1. Eficiencia de memoria: Cada tipo usa una cantidad diferente de bytes
2. Precisión: Algunos tipos pueden almacenar valores decimales, otros solo enteros
3. Validación: El compilador verifica que las operaciones sean válidas para ese tipo
4. Prevención de errores: Evita que se realicen operaciones incompatibles

3. VARIABLES: DECLARACIÓN Y USO

¿Qué es una variable?

Una variable es un espacio de memoria etiquetado con un nombre que almacena un valor. Este valor puede cambiar durante la ejecución del programa. Cada variable debe tener un tipo de dato asignado.

Cómo declarar una variable en PSeInt:

variable tipo: nombre

Ejemplo:

Var Entero edad Real peso Carácter nombre Lógico esmayor

4. LOS 4 TIPOS DE DATOS EN PSEINT

TIPO	PALABRA CLAVE	RANGO	BYTES
Entero	Entero	-2,147,483,648 a 2,147,483,647	4
Real	Real	$\pm 1.7 \times 10^{-308}$ a $\pm 1.7 \times 10^{308}$	8
Carácter (Texto)	Carácter	Texto entre comillas	Variable
Lógico	Lógico	Verdadero / Falso	1

4.1 TIPO ENTERO

Definición: Almacena números completos sin decimales (positivos y negativos).

Rango: -2,147,483,648 a 2,147,483,647

Memoria: 4 bytes

Usos comunes: Edades, cantidades, contador de iteraciones, IDs, años, números de identificación

Ejemplos válidos:

edad <- 25 cantidad <- 100 negativo <- -50 cero <- 0 identificacion <- 98765432

4.2 TIPO REAL

Definición: Almacena números con parte decimal (punto flotante).

Rango: Aproximadamente $\pm 1.7 \times 10^{-308}$ a $\pm 1.7 \times 10^{308}$

Memoria: 8 bytes (el doble que Entero)

Usos comunes: Precios, medidas, promedios, cálculos científicos, temperaturas, pesos, alturas

Ejemplos válidos:

precio <- 49.99 altura <- 1.75 promedio <- 8.5 temperatura <- -15.3 pi <- 3.14159

4.3 TIPO CARÁCTER (CADENA DE TEXTO)

Definición: Almacena texto - secuencias de caracteres (letras, números, símbolos).

Sintaxis: Se escribe entre comillas dobles " "

Longitud: Sin límite práctico

Usos comunes: Nombres, mensajes, direcciones, descripciones, teléfonos, emails

Ejemplos válidos:

```
nombre <- "Juan Pérez" ciudad <- "San José" telefono <- "8765-4321" mensaje <- "Bienvenido a PSeInt" email <- "estudiante@email.com"
```

4.4 TIPO LÓGICO (BOOLEANO)

Definición: Almacena solo dos valores posibles: Verdadero o Falso.

Valores: Verdadero (true) o Falso (false)

Memoria: 1 byte

Usos comunes: Banderas (flags), validaciones, decisiones en condicionales, resultado de comparaciones

Ejemplos válidos:

```
esmayor <- Verdadero esmenor <- Falso validar <- Verdadero encontrado <- Falso esvalido <- (edad >= 18) //  
Resultado de una comparación
```

5. CONSTANTES: DEFINICIÓN Y CARACTERÍSTICAS

¿Qué es una constante?

Una constante es un valor que NO cambia durante la ejecución del programa. Una vez que se asigna un valor a una constante, ese valor permanece fijo.

Declaración de constantes en PSeInt:

```
const tipo nombre = valor
```

Ejemplo:

```
Const Entero CANTIDADMESES = 12 Real PI = 3.14159 Carácter UNIVERSIDAD = "UCR" Lógico  
ESVALIDO = Verdadero
```

Ventajas de usar constantes:

Claridad: El nombre documenta el propósito del valor

Mantenibilidad: Si el valor cambia, se modifica en un solo lugar

Prevención de errores: Evita modificaciones accidentales

Eficiencia: Algunas constantes se optimizan en tiempo de compilación

6. INDICADORES Y REGLAS DE NOMENCLATURA

¿Qué son los indicadores?

Los indicadores son símbolos o marcas que permiten identificar el tipo o propósito de una variable o constante dentro del código.

Convenciones comunes (camelCase y snake_case):

camelCase: Variables normales comienzan con minúscula esnecesario, cantidadTotal, edadPersona

UPPERCASE: Constantes se escriben en MAYÚSCULAS PI, CANTIDADMESES, VELOCIDADLUZ

snake_case: Alternativa con subguiones edad_estudiante, cantidad_total, nombre_ciudad

Reglas de nomenclatura en PSeInt:

Debe comenzar con una letra (a-z, A-Z) o subguión (_)

Puede contener letras, números (0-9) y subguiones (_)

NO se permiten espacios en blanco

NO se permiten caracteres especiales (!, @, #, \$, %, etc)

NO puede ser una palabra reservada (Var, Const, si, mientras, para, etc)

Ser descriptivos y significativos

7. CONVERSIÓN ENTRE TIPOS

A veces necesitamos convertir un valor de un tipo a otro. La conversión de datos entre números y texto (conocida como *type casting* o *parsing*). es una de las operaciones más comunes y críticas en la programación. Por ejemplo en datos de formulario: `<input type="text">`

PSeInt proporciona funciones específicas para esto:

ConvertirATexto(valor) - Convierte número a texto

```
resultado <- ConvertirATexto(123) // Resultado: "123" (tipo Carácter)
```

ConvertirANumero(texto) - Convierte texto a número

```
numero <- ConvertirANumero("456") // Resultado: 456 (tipo Entero)
```

Conversión automática implícita:

Entero a Real: Un Entero se puede usar donde se espera un Real

```
resultado <- 10 + 3.5 // 10 (Entero) se convierte a Real automáticamente
```

⚠ IMPORTANTE: Cuidado con conversiones inválidas

```
ConvertirANumero("abc") // ¡ERROR! No se puede convertir "abc" a número
```

8. EJEMPLOS PRÁCTICOS

Ejemplo 1: Declaración de variables de diferentes tipos

Algoritmo EjemploTipos

```
Var    Entero edad    Real peso    Carácter nombre    Lógico esmayor
```

```
edad <- 25    peso <- 75.5    nombre <- "Carlos"    esmayor <- Verdadero
```

```
Escribir "Edad: ", edad    Escribir "Peso: ", peso, " kg"    Escribir "Nombre: ", nombre    Escribir "¿Es mayor?: ", esmayor
```

FinAlgoritmo

Ejemplo 2: Usando constantes en cálculos

Algoritmo CalculoCirculo

```
Const    Real PI = 3.14159    Var    Real radio, area    radio <- 5.0    area <- PI * radio * radio
```

```
Escribir "Radio: ", radio    Escribir "Área: ", area
```

FinAlgoritmo

Ejemplo 3: Conversión de tipos

Algoritmo ConversionTipos

Var Entero numero Carácter texto

numero <- 123

texto <- ConvertirATexto(numero)

Escribir "Número como entero: ", numero Escribir "Número como texto: ", texto

FinAlgoritmo

9. REFERENCIAS Y FUENTES

1. PSeInt - Software Educativo de Algoritmos y Programación
2. Programa Educativo: Desarrollo de Páginas Web (CSTI14001)
3. Módulo 4: Diseño de Algoritmos (CSTI12010)
4. Knuth, D. E. (1997). The Art of Computer Programming. Addison-Wesley.
5. Deitel, P., & Deitel, H. (2016). C++ How to Program. Pearson.